ASSIGNMENT 6 — QUESTION 10

Q10 - The Final Delivery: Async Pipeline Debugger

JavaScript (ES6+) solution with comments

```
// Q10: Asynchronous food delivery pipeline with random delays/failures.

function runStep(stepName, successMessage) {
    return new Promise((resolve, reject) => {
        const delay = 1000 + Math.floor(Math.random() * 1000);

        setTimeout(() => {
            // Each step has a 15% chance of failure.
            if (Math.random() < 0.15) {
                reject(new Error(stepName + " failed"));
            } else {
                console.log(successMessage);
                resolve();
            }
        }, delay);
    });
}

async function runPipeline() {
    console.log("Start Pipeline");

    try {
        // await keeps the delivery steps in the required order.
        // The timer waits do not block the JavaScript event loop.
        await runStep("takeOrder", "Step 1: Order taken");
        await runStep("prepare", "Step 2: Food prepared");
        await runStep("pack", "Step 3: Package ready");
        await runStep("dispatch", "Step 4: Out for delivery");
        await runStep("deliver", "Delivery completed!");
    } catch (error) {
        // The first rejected step jumps directly to this catch block.
        console.error("Pipeline failed!", error.message);
    }
}

runPipeline();

/*
Event loop notes:
- runStep() returns a Promise that settles later through setTimeout().
- await suspends runPipeline(), not the entire JavaScript thread.
- When a Promise settles, the async function continues as a microtask.
- Because each step is awaited, the next step starts only after the
  previous one succeeds. A rejection skips the remaining steps and
  is handled by try/catch.
*/
```